

GIT & GITHUB

- Professional Hands-on Session
- Version Control | Collaboration | Industry Workflow



SESSION OBJECTIVES

- Understand Version Control
- Learn essential Git commands
- Work with branches
- Collaborate using GitHub
- Handle real-world scenarios



WHAT IS VERSION CONTROL?

- Tracks changes in code over time
- Allows reverting to previous versions
- Enables multiple developers to collaborate



WHAT IS GIT?

- Distributed Version Control System
- Works locally on your machine
- Fast and lightweight
- Industry standard tool



WHY GIT IS IMPORTANT IN REAL WORLD

- Team collaboration
- Track project history
- Code review process
- Rollback production issues



INSTALLING & CONFIGURING GIT

- Check version: `git --version`
- Set username: `git config --global user.name "Your Name"`
- Set email: `git config --global user.email "you@email.com"`



INITIALIZING A REPOSITORY

- Create repo: `git init`
- Check status: `git status`
- Real Example: Starting a new React project



ADDING & COMMITTING CHANGES

- Add file: `git add filename`
- Add all: `git add .`
- Commit: `git commit -m "Add login feature"`
- Commit = Save snapshot of your project



VIEWING HISTORY

- View commits: `git log`
- One-line log: `git log --oneline`
- Real Example: Tracking bug fixes history



UNDERSTANDING BRANCHES

- Branch = Independent line of development
- Main branch = production-ready code
- Feature branches for new features



BRANCH COMMANDS

- Create branch: `git branch feature/login`
- Switch branch: `git switch feature/login`
- Create & switch: `git switch -c feature/signup`
- List branches: `git branch`



MERGING BRANCHES

- Switch to main: `git switch main`
- Merge: `git merge feature/login`
- Real Example: Merge completed feature to production



MERGE CONFLICTS

- Occurs when same line is edited differently
- Open file and resolve manually
- After fixing: `git add .`
- Commit again to complete merge



CONNECTING TO GITHUB

- Add remote: `git remote add origin <repo_url>`
- Push code: `git push -u origin main`
- Clone repo: `git clone <repo_url>`



GIT PULL & FETCH

- `git fetch` → downloads changes only
- `git pull` → fetch + merge
- Real Example: Syncing team updates



PULL REQUESTS (PR)

- Create branch
- Push branch
- Open Pull Request on GitHub
- Team reviews code before merging



FORKING WORKFLOW

- Fork repository
- Clone your fork
- Create branch & make changes
- Submit PR to original repo



GIT STASH (TEMPORARY SAVE)

- `git stash` → save unfinished work
- `git stash pop` → restore changes
- Useful when switching branches quickly



AMENDING LAST COMMIT

- `git commit --amend`
- Used to fix last commit message
- Add forgotten files to last commit



REBASING (CLEAN HISTORY)

- `git rebase main`
- Reapply commits on top of another branch
- Keeps commit history clean (Advanced use)



BEST PRACTICES

- Use meaningful commit messages
- Keep commits small and focused
- Use branch naming conventions
- Always pull before pushing



BRANCH NAMING CONVENTION

- feature/login-ui
- bugfix/navbar-error
- hotfix/payment-crash
- chore/update-dependencies



REAL WORLD TEAM WORKFLOW

- 1. Pull latest main branch
- 2. Create feature branch
- 3. Develop feature
- 4. Push branch
- 5. Create PR
- 6. Review & Merge



COMMON INTERVIEW QUESTIONS

- Difference between Git and GitHub?
- What is merge conflict?
- What is rebase?
- Difference between fetch and pull?



HANDS-ON PRACTICE TASK

- Initialize repo
- Create feature branch
- Add file and commit
- Push to GitHub
- Create PR and merge



FINAL TAKEAWAYS

- Git = Version Control
- GitHub = Collaboration Platform
- Branches = Teamwork
- PR = Code Review Process

